Adedoyin Ahoton — Build Log

I’m a developer who learns by shipping — each cycle I pick a real problem, build a working project end-to-end, and write down what broke along the way. I like tools that save time or make messy data easier to reason about, and I’d rather have four scrappy, functioning builds than one polished thing that never ships. These four cycles were built working through pairs and as part of a team, so a lot of what I learned came from splitting up problems, reviewing each other’s code, and figuring out how to ship together — not just solo. This repo tracks that progress, cycle by cycle.

Cycle 1 — Slack Clone

Status: Shipped

Problem: Team messaging tools like Slack look simple on the surface, but building one surfaces real problems: real-time sync, per-user data scoping, and auth done right instead of faked.

What it does: A Slack-inspired team messaging app. Users sign up and sign in for real, get their own channels and DMs, and see messages update live — no page refresh, no fake data.

Key features: - Real sign up / sign in with Supabase Auth - Channels with membership — add or remove teammates per channel - Channel and direct messages stored in Postgres - Per-user views — each person only sees their own channels and DMs - Real-time message updates via Supabase Realtime - Profile status and presence

Stack: Next.js 15 (App Router), React, TypeScript, Tailwind CSS, Supabase (Auth, Postgres, Realtime)

Live demo: https://brovaset.github.io/Slack-Clone/

Notes: *Realtime updates and per-user data scoping fought each other more than expected — getting each person to see only their own channels/DMs without breaking the live subscription took a few passes.*

Cycle 2 — AdScale AI

Status: Shipped

Data insight: Advertising managers are experiencing a slow, largely invisible displacement. The executional work that defines the role — building, versioning, launching, and reporting on campaigns — is increasingly automated by self-serve ad platforms and absorbed into larger marketing teams. From an identical 2020 baseline, marketing-management jobs grew about 46% while advertising-management jobs fell about 4.5% over five years — a divergence raw headcounts hide, since advertising is roughly 18x smaller as a field. Advertising managers are left doing high-volume, low-leverage busywork with no time for strategic work, and no easy way to prove their impact in the data-fluent language marketing teams now speak.

What it does: An AI advertising manager that turns a plain-language brief into a real, launched ad campaign — with a human approving every dollar before it spends. AdScale lets you describe a campaign in

one sentence, has a Claude agent draft the structure (audience, budget split, ad copy, channels), and only launches through the real ad-platform API after explicit human approval — giving one manager the output of a team.

Key features: - AI campaign generation — Claude parses a plain-language brief into a structured campaign via tool use - Approve-before-spend gate enforced server-side (launch is rejected without explicit approval) - Real campaign creation through the Meta Marketing API (sandbox account, created paused — can't spend real money) - Server-side spend guardrails and approval thresholds the agent can't exceed - AI optimization suggestions in plain language, plus an "indexed impact" performance summary - Google sign-in via Supabase Auth (OAuth-only, no stored passwords)

Stack: React 19 + Vite + Tailwind CSS (frontend), FastAPI + SQLAlchemy + Pydantic (backend, 23 passing contract tests), Claude (Anthropic API) for the agent, Supabase Auth + Postgres, Meta Marketing API / Google Ads API integrations, deployed on Vercel

Live demo: https://adscale-99th-dragon.vercel.app

Built with: a two-person team in parallel branches — backend/AI/integrations on one side, frontend/dashboard/approval flows on the other — during Pursuit's AI-Native Program

Notes: *The approve-before-spend gate had to be airtight before we trusted it with a real API, so we spent more time on that guardrail logic and the mock-mode fallback than on the UI itself.*

Cycle 3 — AccountPulse

Status: In progress

Problem: CSMs experience fragmented account monitoring because renewal dates, product usage, support tickets, and customer communications live across separate systems. A CSM managing ~50 accounts can spend 2-3 hours a day just aggregating account-health signals before deciding whether action is needed — and a common missed-risk scenario is a support ticket open 7+ days on an account also inside a 60-day renewal window, where those two signals simply never get looked at side by side.

What it does: AccountPulse is an AI agent for Customer Success Managers that identifies at-risk customer accounts before risk becomes churn. It's a **read-only** advisor: it runs each morning or on request, pulls signals from CRM, product usage, support, and communication tools, classifies each account (Action Needed / Watch / Healthy / Needs Manual Review), explains its reasoning with sources, and recommends a next step. The CSM reviews the evidence and decides which accounts to act on — the agent informs, the human decides.

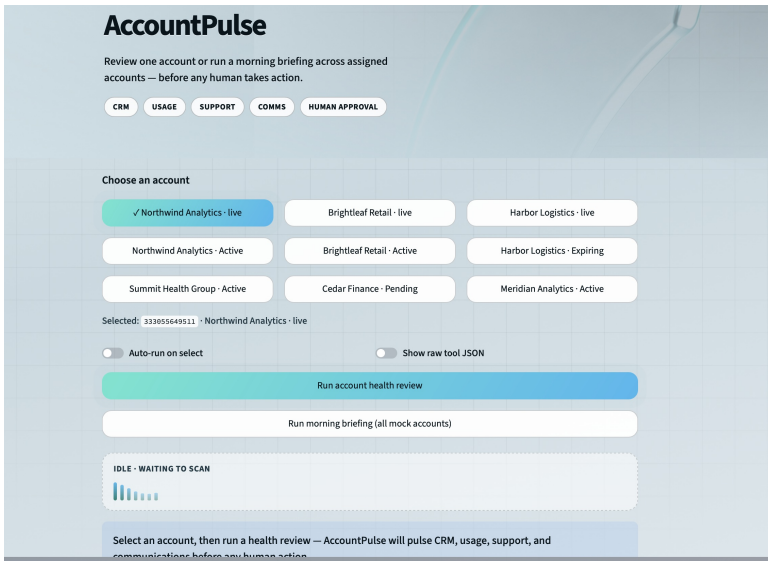
Observe → decide → act loop: AccountPulse observes account signals across CRM, support, usage, and comms → decides a risk classification and next-step recommendation → surfaces a prioritized briefing for the CSM to act on.

Key features: - Four tools pulling account signals: CRM (HubSpot), product usage (PostHog), support tickets (Zendesk), communication activity (Gmail) — mock data by default, live connectors when credentials are set - Deterministic, rules-based reporting (not just LLM output) for stable, reproducible demo results - Morning briefing mode — prioritized report across an entire book of accounts - Human-approval-required guardrail before any customer-facing or account-

changing action - Treats CRM notes/ticket text as untrusted input — explicitly resists prompt injection from within that data - Streamlit UI for picking an account and running a review or a full briefing

Stack: Python, Streamlit, pytest, uv (dependency/env management); mock + live integrations with HubSpot, PostHog, Zendesk, Gmail

Screenshot:



AccountPulse UI — account selection and health review

Choosing an account and running an on-demand health review or a morning briefing across the assigned book.

Built with: a two-person team — one owning CRM/HubSpot integration, Streamlit UI, and fixtures; the other owning the system prompt, agent setup, support/usage/comms tools, and evals

Notes: *One of our mock accounts is a deliberate prompt-injection test — that forced us to design for untrusted CRM/ticket text early instead of bolting it on later.*

Cycle 4 — PopEngine

Status: In progress

Problem: Independent NYC event organizers have to navigate a permit maze spread across at least seven different agencies (SAPO, Parks, NYPD, DOT, FDNY, DOB, Health), each with its own portal, lead time, and fee structure. A single event that touches a sidewalk, serves food, and plays music can trigger four separate permits with lead times ranging from 5 days to over a year out, and nothing tells the organizer up front which permits apply or whether their date is even feasible. Right now the only real alternatives are hiring a production agency (priced for brand activations, not independents) or piecing it together from static blog guides that can’t evaluate a specific event.

What it does: PopEngine takes a short questionnaire about the event (borough, location type, headcount, date, food, sound, structures, flame, alcohol, power) and generates a complete, source-cited permit plan: every required permit and agency, official lead times and fees, required documents, and a backward-computed timeline with an immediate feasibility verdict (Feasible / Feasible-at-risk / Conditional / Infeasible). The plan becomes a live checklist with deadline alerts and direct links to the right city portals through event day.

Features complete so far (MVP core): - Event intake questionnaire with conditional branching — a typical event takes 10-15 questions, under 2 minutes - Permit plan generator evaluating a 33-rule, 4-advisory NYC ruleset, with every line item citing its official source and verification status (confirmed / conflicting / research-required) - Feasibility verdict engine — computes deadlines backward from the event date and flags infeasible timelines with the specific blocking requirement (e.g. a missed 45-day SAPO filing window) - Rules snapshot banner showing exactly which ruleset version and publish date produced the plan - Live compliance checklist with per-permit status tracking and document upload - Deadline alerts (email live; SMS simulated in-product as a fallback until Twilio registration clears) - Portal deep links with prepared document packages for each required permit

Still in progress: App-less QR check-in and a live ops dashboard for event day (Phase 1.5 stretch goals), plus a longer roadmap covering saved/resumable intake, full application tracking, calendar export, AI-assisted free-text intake, and eventually rules support for cities beyond NYC.

Stack: React / Next.js (frontend), Node.js / Express (backend), PostgreSQL (including a dedicated rules table), Twilio for SMS deadline alerts. Rules are stored as versioned data (conditions → findings) rather than hardcoded logic, so updates to NYC permit requirements are data changes, not code changes.

Screenshot / live link: *[placeholder — add a screenshot of the organizer UI or the gated demo link]*

Built with: a four-person team — Naquan McKune, Jason Zeng, Adedoyin Ahoton, and Bo Moldenhauer

Notes: *Keeping the rules engine pure (no DB/HTTP/clock) made testing easy but meant extra plumbing to wire real dates and data back in at the API layer.*

Last updated: July 25, 2026